Distributed Systems Mini Project – Cases/Scenarios

Example (1): Media Sharing Platform

Description: Teams in this scenario build a simple media-sharing service (photo or video). Users can upload media files, which the system stores and processes (e.g., generate thumbnails). Users can then browse or download the media.

Features:

- **Upload Media:** A client (web or CLI) sends an upload request (with a file and metadata: title, description) to the API. The API generates a storage URL (or directly uploads) and stores the file in **Object Storage**.

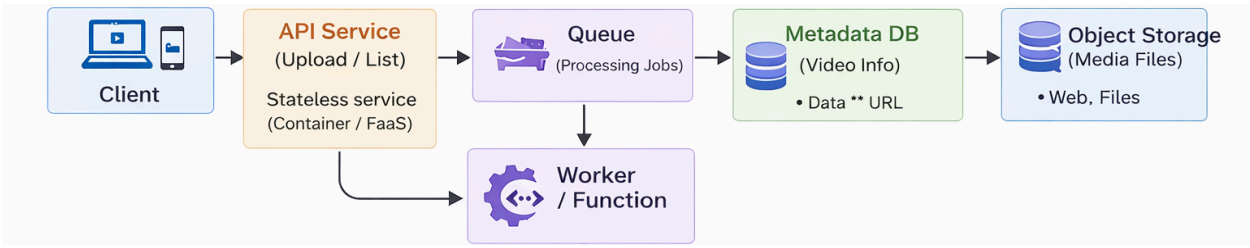
- **Metadata Record:** The API creates a metadata entry in the database with fields: media_id (UUID), title, uploader, object_key, status ("pending"), thumbnail_key, timestamp.

- **Processing Worker:** After upload, the API pushes a message to the **Message Queue** with {media_id, object_key}. A worker function (triggered by the queue) downloads the media from storage, generates a thumbnail (for images/videos), saves the thumbnail back to the bucket, and updates the DB record (thumbnail_key and status="done").

- **List/Fetch Media:** The API provides endpoints /list to list media (titles and thumbnails) and /media?id=... to retrieve a specific file or its metadata. The client can display thumbnails and let users click to view/download the full media (using the stored object key).

- **Optional (Extra Credit):** Support EXIF metadata extraction, video transcoding, or convert image formats. Ensure the pipeline is fault-tolerant (e.g. skip files > size limit).

Architecture (Rough Sketch):



- **Data Model:**
```
{
 "media_id": "string (UUID)",
 "title": "string",
 "uploader": "string",
 "object_key": "string (storage key/URL)",
 "status": "pending|done|error",
 "thumbnail_key": "string (storage key/URL)",
 "created_at": "timestamp"
}
```

- **API Endpoints (example):**
  - POST /upload with form-data: file, title, uploader. Returns {"media_id": "...", "status": "pending"}.
  - GET /list returns a list of all media metadata (id, title, thumbnail URL if done).
  - GET /status?id=... returns metadata for one media_id.
  - (Optional) GET /media?id=... generates a signed URL or proxies the file.

- **Implementation Notes:** Use image/video libraries (e.g. PIL in Python, ffmpeg) to create thumbnails. For local prototype, store files in ./storage/; in cloud, use S3 (and S3 event can also trigger Lambda as alternative to queue).

Deliverables: A working media service where users can upload images/videos and then see them listed with thumbnails when processing completes.

Example (2): IoT Data Aggregation

Description: Build a system to collect and process IoT sensor data (e.g. environmental readings). Simulate multiple sensors sending data. The system should store raw sensor data and compute summaries.

Features:

- **Sensor Upload:** Each sensor (or simulated device) posts data (e.g. temperature/humidity readings) to the API. Each upload might be a JSON object or a file (e.g. JSON or CSV) representing a data log for a period.

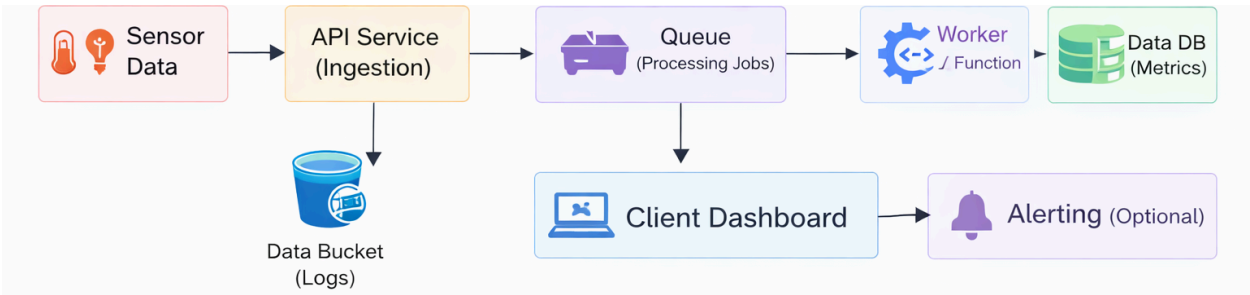
- **Raw Data Storage:** The API stores the raw data in **Object Storage** (e.g. each upload creates a JSON/CSV file in a bucket) and records a metadata entry: data_id, sensor_id, object_key, status, timestamp.

- **Asynchronous Processing:** The API enqueues a job with {data_id, object_key}. A worker function retrieves the data file from storage, parses it, computes summary statistics (e.g. max/min/avg), and writes results to the DB (update the record with summary and status="done").

- **Query Interface:** The API provides endpoints to list data uploads and query summaries. For example, /summaries returns summary stats for each sensor or time period.

- **Live Alerts (optional):** If a reading exceeds a threshold, the worker could publish an alert message (e.g. print to logs or send to another queue) – as an advanced feature.

Architecture (Rough Sketch):



- **Data Model:**

```
{
 "data_id": "UUID",
 "sensor_id": "string",
 "object_key": "string",
 "status": "pending|done",
 "summary": { "min": number, "max": number, "avg": number },
 "timestamp": "time of upload"
}
```

- **API Endpoints:**
  - POST /data with JSON body: { "sensor_id": "S1", "values": [..] }. Returns {"data_id": "...", "status": "pending"}. (Alternatively, support file upload).
  - GET /list returns all data IDs and sensor IDs.
  - GET /summary?id=... returns the summary statistics for that data_id (after processing).

- **Implementation Notes:** For simulation, you can generate random numbers. Worker should parse JSON or CSV and compute stats. Use a small dataset to stay within free-tier. DynamoDB or Cosmos can store summary fields.

Deliverables: A system where simulated sensors push data and then later retrieve computed summaries through the API.

Example (3): Game Score Processing

Description: Create a backend for a multiplayer game leaderboard. Players submit game results; the system updates global scores. Optionally handle an uploaded game replay file.

Features:

- **Submit Score:** Clients (players) send game results to the API, including player_name, score, and optionally a replay file (binary blob).

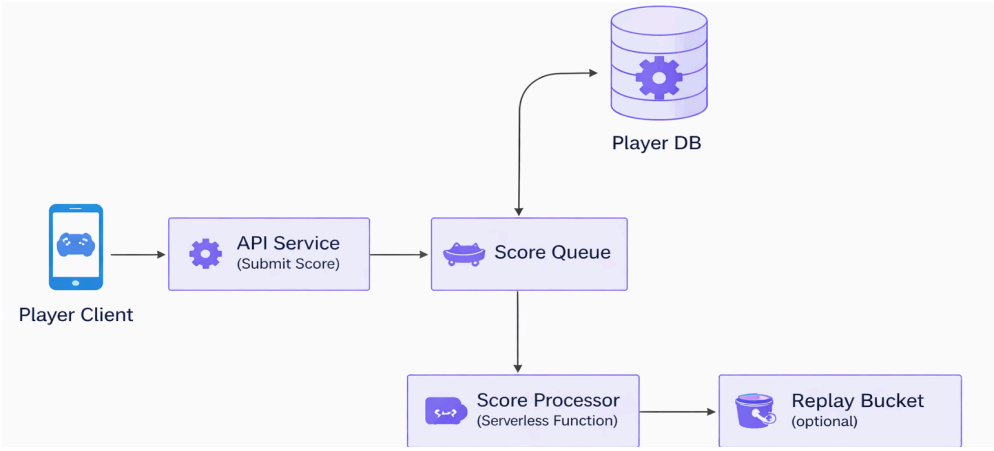
- **Queue & Worker:** The API enqueues a job with {player_name, score, object_key (if replay)}. A worker function updates a global leaderboard stored in DB (e.g. incrementing total score or ranking), and if a replay file was uploaded, it saves it to object storage.

- **Leaderboard:** Store a table of players with total_score and other stats. The DB record could be: {player_name, total_score, games_played}. The worker updates these fields.

- **Retrieve Rankings:** The API endpoint /leaderboard returns top N players by score. /player?name=... returns stats for one player.

- **Object Storage (Optional):** If replay files are used, API stores them in object storage, and DB holds the object_key. (Clients could download replays via an endpoint).

Architecture (Rough Sketch):



- **Data Model:**
  - **Leaderboard (DB table):**

    {
    "player_name": "string",
    "total_score": number,
    "games_played": number
    }
    (Player name is key.)

  - **Uploads (if replay used):**

    { "upload_id": "UUID",
    "player_name",
    "object_key", "status"
    }

- **API Endpoints:**
  - POST /submit with JSON: { "player": "Alice", "score": 150 } (plus optional file upload). Returns status.
  - GET /leaderboard returns array of top 10 players {player, total_score}.
  - GET /player?name=Alice returns stats {player, total_score, games_played} and link to replay if available.

- **Implementation Notes:** If not using replay files, you can ignore object storage and focus on queue+DB. Ensure race conditions are handled (two players submitting simultaneously). Use DynamoDB conditional writes or transactions if needed.

Deliverables: A functioning scoreboard service: players' scores accumulate, and the leaderboard is updated asynchronously.